



四川大学
SICHUAN UNIVERSITY

操作系统习题课

Exercise class of operating system

四川大学 网络空间安全学院 2024.06





四川大學
SICHUAN UNIVERSITY

01

信号量

“ 信号量”

信号量是在多线程环境下使用的一种设施，可以用来保证两个或多个关键代码段不被并发调用。在进入一个关键代码段之前，线程必须获取一个信号量；一旦该关键代码段完成了，那么该线程必须释放信号量。其它想进入该关键代码段的线程必须等待直到第一个线程释放信号量。

”

Pseudo code



```
struct semaphore {
    int count;
    queueType queue;
};
void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process */;
    }
}
void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```

信号量是一个与等待队列有关的整型变量。

在伪代码中可以看到，有两个对信号量的count值（可用资源数目，count=n）和阻塞队列的操作：semWait&semSignal，semWait可以理解为申请资源；semSignal可以理解为释放资源。

$n > 0$ ：当前有可用资源，可用资源数量为n

$n = 0$ ：资源都被占用，可用资源数量为0

$n < 0$ ：资源都被占用，且还有n个进程排队

“ 问题描述

系统中有一组生产者进程和一组消费者进程，生产者进程每次生产一个产品放入缓冲区，消费者进程每次从缓冲区中取出一个产品并使用。（“产品”实际是某种数据）

生产者、消费者共享一个初始为空、大小为 N 的缓冲区。只有缓冲区没满时，生产者才能把产品放入缓冲区，否则必须等待。只有缓冲区不空时，消费者才能从中取出产品，否则必须等待。缓冲区是临界资源，各进程必须互斥地访问。

”

分析

有两类进程，即生产者和消费者进程

题面是一个混合关系，既有间接关系，也有直接关系：

直接关系：生产者和消费者不能同时进入临界区；

间接关系：缓冲区满，生产者不能放入；缓冲区空，消费者不能取出

生产者-消费者问题



初始化

```
semaphore mutex=1; //标记缓冲区是否可以使用  
semaphore full=N; //缓冲区可以存放N个产品  
semaphore empty=0; //刚刚开始没有产品
```

生产者-消费者问题



```
int main() {  
    parbegin(producer, consumer);  
}
```

```
void producer()  
{  
    while(1) {  
        produce(product);  
        semwait(full);  
        semwait(mutex);  
        put(product);  
        semsignal(mutex);  
        semsignal(empty);  
    }  
}
```

```
void consumer()  
{  
    while(1) {  
        semwait(empty);  
        semwait(mutex);  
        get(product);  
        semsignal(mutex);  
        semsignal(full);  
    }  
}
```

代码描述

“ 问题描述

有一个文件，被若干个读者和若干个写者共同访问，要求：

- 1) 允许多个读者同时访问一个文件；
- 2) 有写者在写文件时，不允许读者访问；

”

分析

两类进程：读者与写者，读者与写者之间的关系是直接关系，但是为了判断“特殊的读者”，还要多考虑一个关系。

为什么要进行这个特殊的判断？

读者-写者问题



初始化

```
semaphore mutex1=1; //文件互斥  
semaphore mutex2=1; //读进程数目互斥
```

读者-写者问题

```
int main() {  
    parbegin(reader, writer);  
}
```

```
void writer()  
{  
    while(1){  
        semwait(mutex1);  
        write(file);  
        semsignal(mutex1);  
    }  
}
```

代码描述

```
void reader()  
{  
    int temp;  
    while(1) {  
        semwait(mutex2);  
        readnum++;  
        temp=readnum;  
        semsignal(mutex2);  
        if (1==temp) {  
            semwait(mutex1);  
            read(file);  
        }  
        semwait(mutex2);  
        readnum--;  
        temp=readnum;  
        semsignal(mutex2);  
        if (0==temp)  
            semsignal(mutex1);  
    }  
}
```



“ 问题描述

有三个进程：Read、Move和Print，共享两个缓存B1和B2

- 1) 进程Read：读取一条记录，并放在缓存B1中
- 2) 进程Move：从缓存B1中读取记录，处理后放入缓存B2中
- 3) 进程Print：从B2中读取数据并打印

请基于信号量相关的操作填空

”

某年OS期末考试真题



Initialize↵		
Semaphore S0 = 1;↵ Semaphore S1 = 0;↵ Semaphore S2 = _____;(1)↵ Semaphore S3 = _____;(2)↵ ↵		
Read Process↵	Move Process↵	Print Process↵
char x;↵ while (true) ↵ {↵ Read a record to x;↵ _____;(3)↵ B1 = x;↵ _____;(4)↵ }↵	char x, y;↵ while (true) ↵ {↵ _____;(5)↵ x = B1;↵ _____;(6)↵ Process x, ↵ store the result to y;↵ _____;(7)↵ B2 = y;↵ _____;(8)↵ }↵ ↵	char x;↵ while (true) ↵ {↵ _____;(9)↵ x = B2;↵ _____;(10)↵ Print x;↵ }↵ ↵

分析

三类进程，两个临界资源：三类进程之间的关系是直接关系，但是需要注意这一点：系统运行时，初始状态B1和B2缓冲中的数据是无效数据。

因此必须保证READ优先使用B1缓存，MOVE优先使用B2缓存

对于READ进程：在对B1缓冲区读取数据时，首先要判断MOVE进程是否将上一次读的数据取走，如果没有取走，READ等待；否则，读取一段数据放到B1，然后通知MOVE进程，B1缓存可用；

对于MOVE进程：首先判断B1缓冲区中的数据是否可用，如果没有等待；否则，从B1缓存中读取数据，然后对数据进行加工，加工完毕后，判断PRINT进程是否已将上次MOVE进程放到B2缓冲区中的数据取走，如果没有取走，等待；否则，将加工后的数据存放到B2缓冲区，然后通知PRINT进程可以使用B2缓冲区；

对于PRINT进程：判断B2缓冲区的数据是否可用，如果不可用，则等待；如果可用，则打印，然后通知MOVE进程，B2的数据已取走，MOVE进程可对B2缓冲区进行更新操作。

初始化

通过上面分析，可以知道，在这个程序中我们需要使用四个信号量，其中：

S1用于表明READ是否可以使用B1；（B1中是否有空位置）

S2用于表明MOVE是否可以使用B1；（B1中是否有数据）

S3用于表明MOVE是否可以使用B2；（B2中是否有有位置）

S4用于表明PRINT是否可以使用B2；（B2中是否有数据）

由于优先级的问題，所以有

Semsignal s1=1；（B1中有位置）1表示有位置，0表示没有位置

Semsignal s2=0；（B1中没有数据）1表示有数据，0表示没有数据

Semsignal s3=1；（B2中有位置）1表示有位置，0表示没有位置

Semsignal s4=0；（B2中没有数据）1表示有数据，0表示没有数据

某年OS期末考试真题



最终答案

Read Process	Move Process	Print Process
<pre>char x; while (true) { Read a record to x; <u>Semwait(s1);(3)</u> B1 = x; <u>Semsignal(s2);(4)</u> }</pre>	<pre>char x, y; while (true) { <u>Semwait(s2);(5)</u> x = B1; <u>semsignal(s1);(6)</u> Process x, store the result to y; <u>semwait(s3);(7)</u> B2 = y; <u>Semsignal(s4);(8)</u> }</pre>	<pre>char x; while (true) { <u>Semwait(s4);(9)</u> x = B2; <u>semsignal(s3);(10)</u> Print x; }</pre>

semWait & semSignal

- semWait(S): 请求分配一个资源
- semSignal(S): 释放一个资源。
- semWait、semSignal操作必须成对出现。

Mutual Exclusion and Synchronization

- 信号量用于互斥时，位于同一进程内（初始值一般为1）
- 用于同步时，交错出现于两个合作进程内——在前事件后加semSignal，在后事件前加semWait
- 注意多个信号量操作的次序，颠倒的顺序可能导致死锁



四川大學
SICHUAN UNIVERSITY

02

银行家算法

页面置换算法



死锁的可能性	死锁的存在性
1. 互斥	1. 互斥
2. 不可抢占	2. 不可抢占
3. 占有且等待	3. 占有且等待
	4. 循环等待

银行家算法核心思想：在进程提出资源申请时，先预判此分配是否会导致系统进入不安全状态。如果会进入不安全状态，就暂时不答应这次请求，让该进程先阻塞等待。

WHAT

原 则	资源分配策略	不同的方案	主要优点	主要缺点
预防	保守：预提交资源	一次性请求所有资源	- 对执行一连串活动的进程非常有效 - 不需要抢占	- 低效 - 延迟进程的初始化 - 必须知道将来的资源请求
		抢占	用于状态易于保存和恢复的资源时非常方便	过于经常地、没必要地抢占
		资源排序	- 通过编译时检测是可以实施的 - 既然问题已在系统设计时解决，因此不需要在运行时计算	禁止增加的资源请求
避免	介于检测和预防中间	操作以便发现至少一条安全路径	不需要抢占	- 必须知道将来的资源请求 - 进程不能被长时间阻塞
检测	非常自由：只要有可能，请求的资源都允许	周期性地调用以便测试死锁	- 不会延迟进程的初始化 - 易于在线处理	固有的抢占丢失

银行家算法



可用

R1	R2	R3	R4
2	1	0	0

进程	当前分配				最大需求				仍然需要			
	r1	r2	r3	r4	r1	r2	r3	r4	r1	r2	r3	r4
P1	0	0	1	2	0	0	1	2				
P2	2	0	0	0	2	7	5	0				
P3	0	0	3	4	6	6	5	6				
P4	2	3	5	4	4	3	5	6				
P5	0	3	3	2	0	6	5	2				

1. 计算每个进程仍然可能需要的资源;
2. 系统当前是处于安全状态还是不安全状态?
3. 系统当前是否死锁, 为什么?
4. 哪个进程 (如果存在) 是死锁的或可能变成死锁的?
5. 如果P3的请求 (0, 1, 0, 0) 到达, 是否可以立即安全地同意该请求? 如果立即同意全部请求, 哪个进程 (如果有) 是死锁的或可能变成死锁的。

知识点: 资源分配、安全状态定义、死锁避免、死锁检测算法

1. 计算每个进程仍然可能需要的资源：仍然需要=最大需求-当前分配

可用			
R1	R2	R3	R4
2	1	0	0

进程	当前分配				最大需求				仍然需要			
	r1	r2	r3	r4	r1	r2	r3	r4	r1	r2	r3	r4
P1	0	0	1	2	0	0	1	2	0	0	0	0
P2	2	0	0	0	2	7	5	0	0	7	5	0
P3	0	0	3	4	6	6	5	6	6	6	2	2
P4	2	3	5	4	4	3	5	6	2	0	0	2
P5	0	3	3	2	0	6	5	2	0	3	2	0

2. 系统当前是处于安全状态还是不安全状态：资源分配拒绝策略-银行家算法

安全状态：指至少有一个资源分配序列不会导致死锁，即所有进程都能运行直到结束。

运行银行家算法，进程结束顺序为：P1->P4->P5->P2->P3时处于安全状态

- P1结束时： $A = (2\ 1\ 0\ 0) + (0\ 0\ 1\ 2) = (2\ 1\ 1\ 2)$
- P4结束时： $A = (2\ 1\ 1\ 2) + (2\ 3\ 5\ 4) = (4\ 4\ 6\ 6)$
- P5结束时： $A = (4\ 4\ 6\ 6) + (0\ 3\ 3\ 2) = (4\ 7\ 9\ 8)$
- P2结束时： $A = (4\ 7\ 9\ 8) + (2\ 0\ 0\ 0) = (6\ 7\ 9\ 8)$
- P3结束时： $A = (6\ 7\ 9\ 8) + (0\ 0\ 3\ 4) = (6\ 7\ 12\ 12)$

3, 4. 系统当前是否死锁：死锁检测算法P181

是一个标记未死锁进程的过程。

可用			
R1	R2	R3	R4
2	1	0	0

1. 标记 Allocation 矩阵中一行全为零的进程。
2. 初始化一个临时向量 W ，令 W 等于 Available 向量。
3. 查找下标 i ，使进程 i 当前未标记且 Q 的第 i 行小于等于 W ，即对所有的 $1 \leq k \leq m$, $Q_{ik} \leq W_k$ 。
若找不到这样的行，终止算法。
4. 若找到这样的行，标记进程 i ，并把 Allocation 矩阵中的相应行加到 W 中，即对所有的 $1 \leq k \leq m$ ，令 $W_k = W_k + A_{ik}$ 。返回步骤 3。

如果所有进程都被标记，则不会死锁；反之则会死锁，未标记的进程都是死锁的。

3, 4. 系统当前是否死锁：死锁检测算法

可用			
R1	R2	R3	R4
2	1	0	0

进程	当前分配				仍然需要			
	r1	r2	r3	r4	r1	r2	r3	r4
P1	0	0	1	2	0	0	0	0
P2	2	0	0	0	0	7	5	0
P3	0	0	3	4	6	6	2	2
P4	2	3	5	4	2	0	0	2
P5	0	3	3	2	0	3	2	0

选择P1, available = (2 1 1 2)

选择P4, available = (4 4 6 6)

选择P5, available = (4 7 9 8)

选择P2, available = (6 7 9 8)

最后P3满足, 所有进程都被标记, 无死锁

5.如果P3的请求 (0, 1, 0, 0) 到达, 是否可以立即安全地同意该请求? 如果立即同意全部请求, 哪个进程 (如果有) 是死锁的或可能变成死锁的。先假设同意P3的请求, 接着计算使用剩下的资源是否能够找到一个进程序列不会造成死锁。

进程	当前分配				最大需求				仍然需要			
	r1	r2	r3	r4	r1	r2	r3	r4	r1	r2	r3	r4
P1	0	0	1	2	0	0	1	2	0	0	0	0
P2	2	0	0	0	2	7	5	0	0	7	5	0
P3	0	1	3	4	6	6	5	6	6	5	2	2
P4	2	3	5	4	4	3	5	6	2	0	0	2
P5	0	3	3	2	0	6	5	2	0	3	2	0

假设同意改请求, 则 $available = (2\ 0\ 0\ 0)$

1.P1结束时, $available=(2\ 0\ 1\ 2)$

2.P4结束时, $available=(4\ 3\ 6\ 6)$

3.P5结束时, $available=(4,6,9,8)$

4.P2和P3无法运行, 造成死锁



四川大學
SICHUAN UNIVERSITY

03

页面置换算法

WHAT

“

进程所要访问的页面不在内存中，就需要把这个不存在的页面调入内存，但内存已经没有空闲空间了，就要求系统从内存中调出一个页面，将其移入磁盘的对换区中。将哪个页面调出来，就要通过算法来确定。把选择换出页面的算法就叫做**页面置换算法**。

好的页面置换算法应具有较低的页面置换概率，并且：

- 将那些以后不再会访问的页面换出；
- 把那些在较长时间内不会被访问的页面换出；

”

例题



题目：

假定系统给某进程分配了三个物理块，页面号引用串为

“7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1”

不同算法：OPT最优算法、LRU、FIFO、Clock算法。

计算：页面置换次数，缺页次数，最优算法等

最佳置换算法OPT



理论上的算法，可以用最佳置换算法OPT去评价其他算法。

思想：最佳置换算法是一种理论上的算法，目前该算法还无法实现，但我们可以用最佳置换算法OPT去评价其他算法；

缺页率：最低；

缺点：无法实现；

步骤	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
页面号	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	
物理块1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7	
物理块2		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0	
物理块3			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1	
缺页	○	○	○	○		○		○			○			○				○			9
置换				○		○		○			○			○				○			6

先进先出页面置换算法FIFO



总是淘汰最先进入内存的页面，就是淘汰掉在内存中**驻留时间最久**的页面。

原理：近期调入的页面被再次访问的概率要大于早期调入页面的概率。

优点：实现简单。

缺点：性能较差：页面调入的先后顺序并不能保证页面使用频率和时间长短。

步骤	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
页面号	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	
物理块1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7	
物理块2		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0	
物理块3			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1	
缺页	○	○	○	○		○	○	○	○	○	○			○	○			○	○	○	15
置换				○		○	○	○	○	○	○			○	○			○	○	○	12

看哪个页面连续出现的次数最长，连续出现最长的页面就是存在最久的页面。

最近最少使用页面算法LRU



每次选择内存中**离当前时刻最久未使用过**的页面淘汰。

优点：性能较接近OPT算法；

缺点：

- 算法效率不高：需要对整个页表频繁的维护，经常用到比较，页面较多时会消耗大量时间。
- 实现要依托较多的硬件支持，实现所需成本较高。

步骤	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
页面号	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	
物理块1（顶）			1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	
物理块2		0	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	
物理块3（底）	7	7	7	0	1	2	2	3	0	4	2	2	0	3	3	1	2	0	1	7	
缺页	○	○	○	○		○		○	○	○	○			○		○		○			12
置换				○		○		○	○	○	○			○		○		○			9

Clock置换算法



当一个页面第一次加载到内存中时，使用位置为1

当页面被引用时，使用位就置为1（简单讲：只要被进程访问，使用位就置位1）

当需要替换页面时，第一个使用位设置为0的帧将被替换

在寻找替换的过程中，设置为1的每个use位都会变为0

被访问到，使用位也还是标记成1。

移到下一位。

步骤	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
页面号	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	
物理块1	7*	7*	7*	2*	2*	2*	2*	4*	4*	4*	4	3*	3*	3	3	0*	0*	0	0*	0*	
物理块2		0*	0*	0	0*	0	0*	0	2*	2*	2	2	2*	1*	1*	1*	1*	7*	7*	7*	
物理块3			1*	1	1	3*	3*	3	3	3*	0*	0*	0*	0	2*	2*	2*	2	2	1*	
缺页	○	○	○	○		○		○	○		○	○		○	○	○		○		○	14
置换				○		○		○	○		○	○		○	○	○		○		○	11

改进型CLOCK算法

思想：在将一个页面换出的时候，如果该页面已经被修改，则要将该页面重新回写到磁盘，如果该页面未被修改，则不需要将它拷回磁盘，就是说，如果这个页面在置换之前已经被修改过了，相比未修改过的页面需要花费**更大的开销**。

过程：除了访问位，再添加一个修改位： $(0,0)$ 、 $(0,1)$ 、 $(1,0)$ 、 $(1,1)$ 。

- 从指针当前的位置开始寻找主存中为 $(0,0)$ 的页面；
- 如果第1步没有找到满足条件的，接着寻找状态为 $(0,1)$ 页面；
- 如果依然没有找到，指针回到最初的位置，将集合中所有页面的使用位设置成0。重复第1步，并且如果有必要，重复第2步，这样一定可以找到将要替换的页面。



四川大學
SICHUAN UNIVERSITY

04

进程调度算法

WHAT

“

进程调度算法也称 CPU 调度算法，当 CPU 空闲时，操作系统就从就绪队列中按照一定的算法选择某个就绪状态的进程，并给其分配 CPU。有以下常见的调度算法。

- 1、First In First Out(FIFO 算法)
- 2、round-robin(轮转算法)
- 3、Shortest Process First (最短进程优先)
- 4、Shortest Remaining Time Next (最短剩余时间)
- 5、Highest Response Ratio Next (最高响应比)

”

例子



进程	A	B	C	D	E	
到达时间	0	2	4	6	8	
服务时间 (T_s)	3	6	4	5	2	平均值
FCFS						
完成时间	3	9	13	18	20	
服务时间 (T_s)	3	7	9	12	12	8.60
T_f/T_s	1.00	1.17	2.25	2.40	6.00	2.56
RR $q = 1$						
完成时间	4	18	17	20	15	
服务时间 (T_s)	4	16	13	14	7	10.80
T_f/T_s	1.33	2.67	3.25	2.80	3.50	2.71
RR $q = 4$						
完成时间	3	17	11	20	19	
服务时间 (T_s)	3	15	7	14	11	10.00
T_f/T_s	1.00	2.5	1.75	2.80	5.50	2.71
SPN						
完成时间	3	9	15	20	11	
服务时间 (T_s)	3	7	11	14	3	7.60
T_f/T_s	1.00	1.17	2.75	2.80	1.50	1.84
SRT						
完成时间	3	15	8	20	10	
服务时间 (T_s)	3	13	4	14	2	7.20
T_f/T_s	1.00	2.17	1.00	2.80	1.00	1.59

例题

题目：

考虑以下的进程集合

进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9

给出不同算法的的进程执行分析

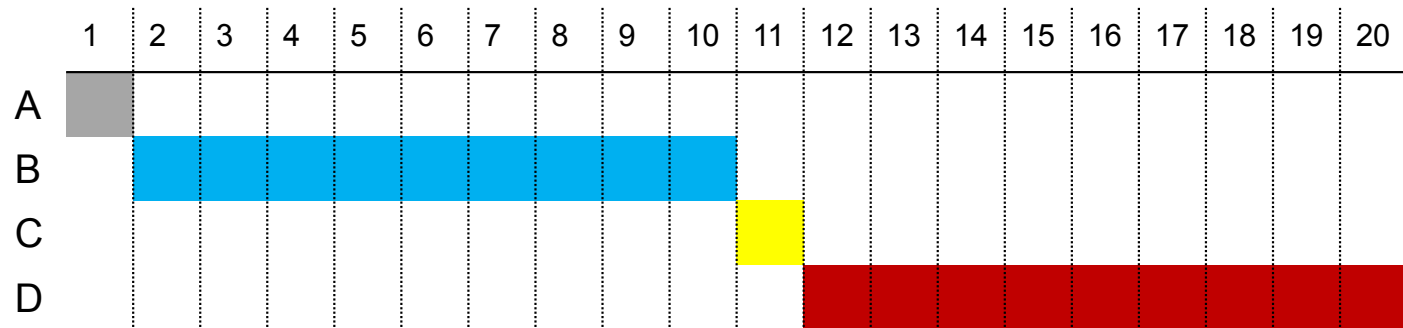
先来先服务算法FCFS



先来先服务最简单的策略是先来先服务 (FCFS),也称为**先进先出**(First-In-First-Out, FIFO)或严格排队方案。每个进程就绪后, 会加入就绪队列。当前正运行的进程停止执行时, 选择就绪队列中存在时间最长的进程运行。

与短进程相比, FCFS更适用于长进程。

进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9



A

← 进程等待队列

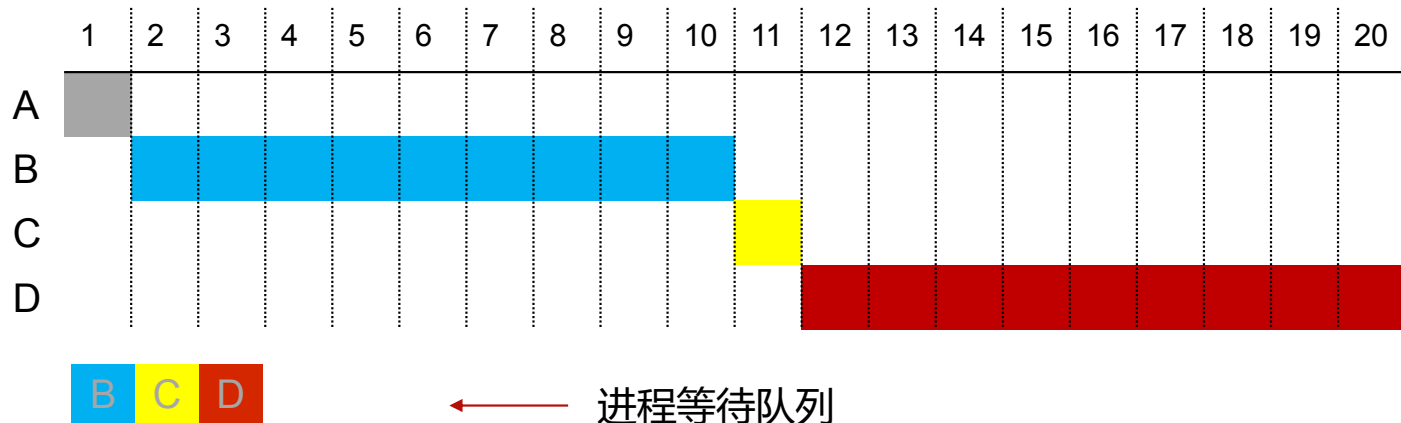
先来先服务算法FCFS



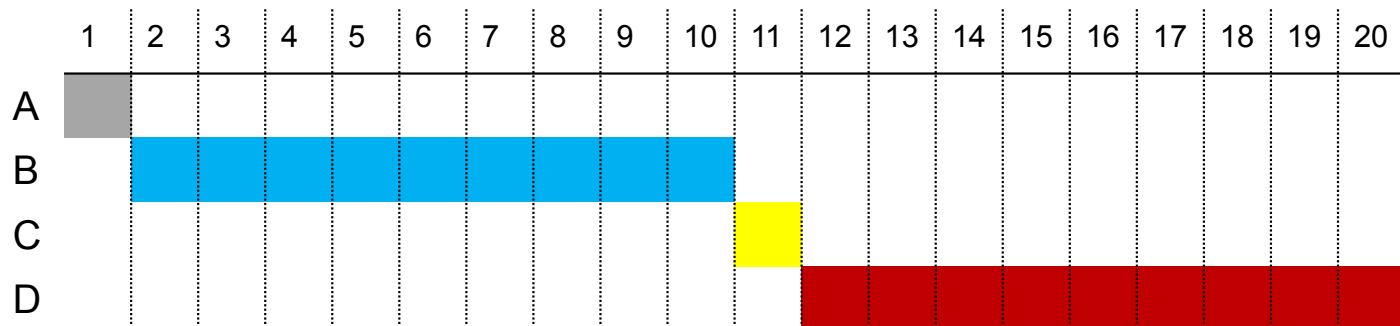
先来先服务最简单的策略是先来先服务 (FCFS),也称为**先进先出**(First-In-First-Out, FIFO)或严格排队方案。每个进程就绪后, 会加入就绪队列。当前正运行的进程停止执行时, 选择就绪队列中存在时间最长的进程运行。

与短进程相比, FCFS更适用于长进程。

进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9



先来先服务算法FCFS



		A	B	C	D	
	T_a	0	1	2	3	
	T_s	1	9	1	9	
FCFS	T_f	1.00	10.00	11.00	20.00	
	T_r	1.00	9.00	9.00	17.00	9.00
	T_r/T_s	1.00	1.00	9.00	1.89	3.22

时间片轮转RR

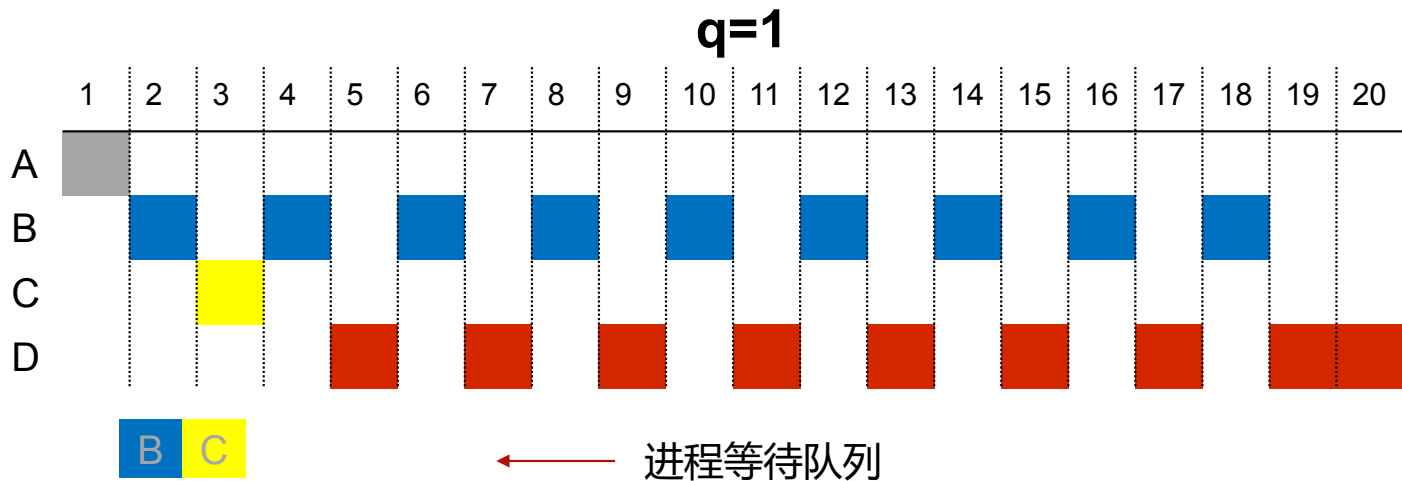


轮转减少FCFS 策略不利于短作业的一类简单方法是，采用基于**时钟的抢占策略**。在这类方法中，最简单的是**轮转算法**。

这种算法周期性地产生时钟中断，出现中断时，当前正运行的进程会放置到就绪队列中，然后基于FCFS 策略选择下一个就绪作业运行。

轮转法最主要的设计问题是所用的时间段(片) 长度。

进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9



时间片轮转RR

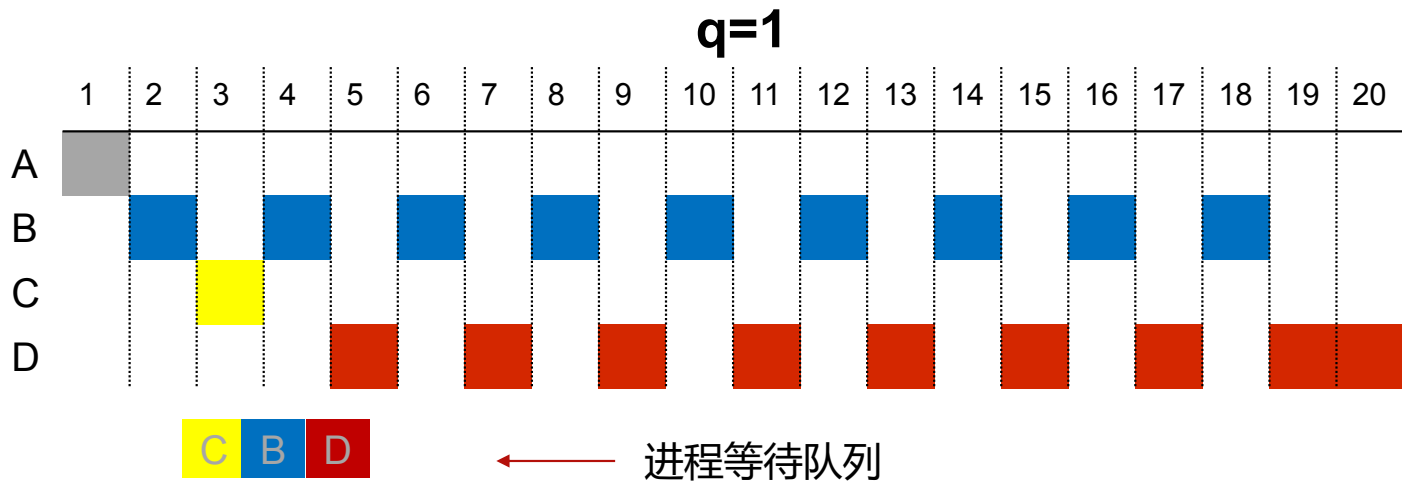


轮转减少FCFS 策略不利于短作业的一类简单方法是，采用基于**时钟的抢占策略**。在这类方法中，最简单的是**轮转算法**。

这种算法周期性地产生时钟中断，出现中断时，当前正运行的进程会放置到就绪队列中，然后基于FCFS 策略选择下一个就绪作业运行。

轮转法最主要的设计问题是所用的时间段(片) 长度。

进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9



时间片轮转RR

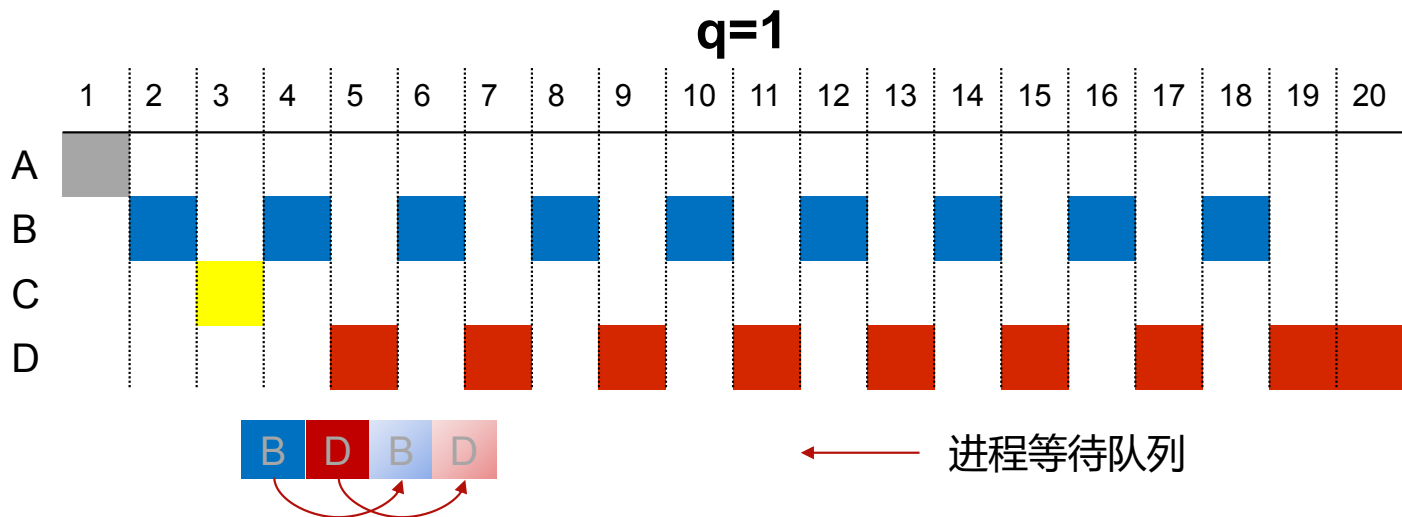


轮转减少FCFS 策略不利于短作业的一类简单方法是，采用基于**时钟的抢占策略**。在这类方法中，最简单的是**轮转算法**。

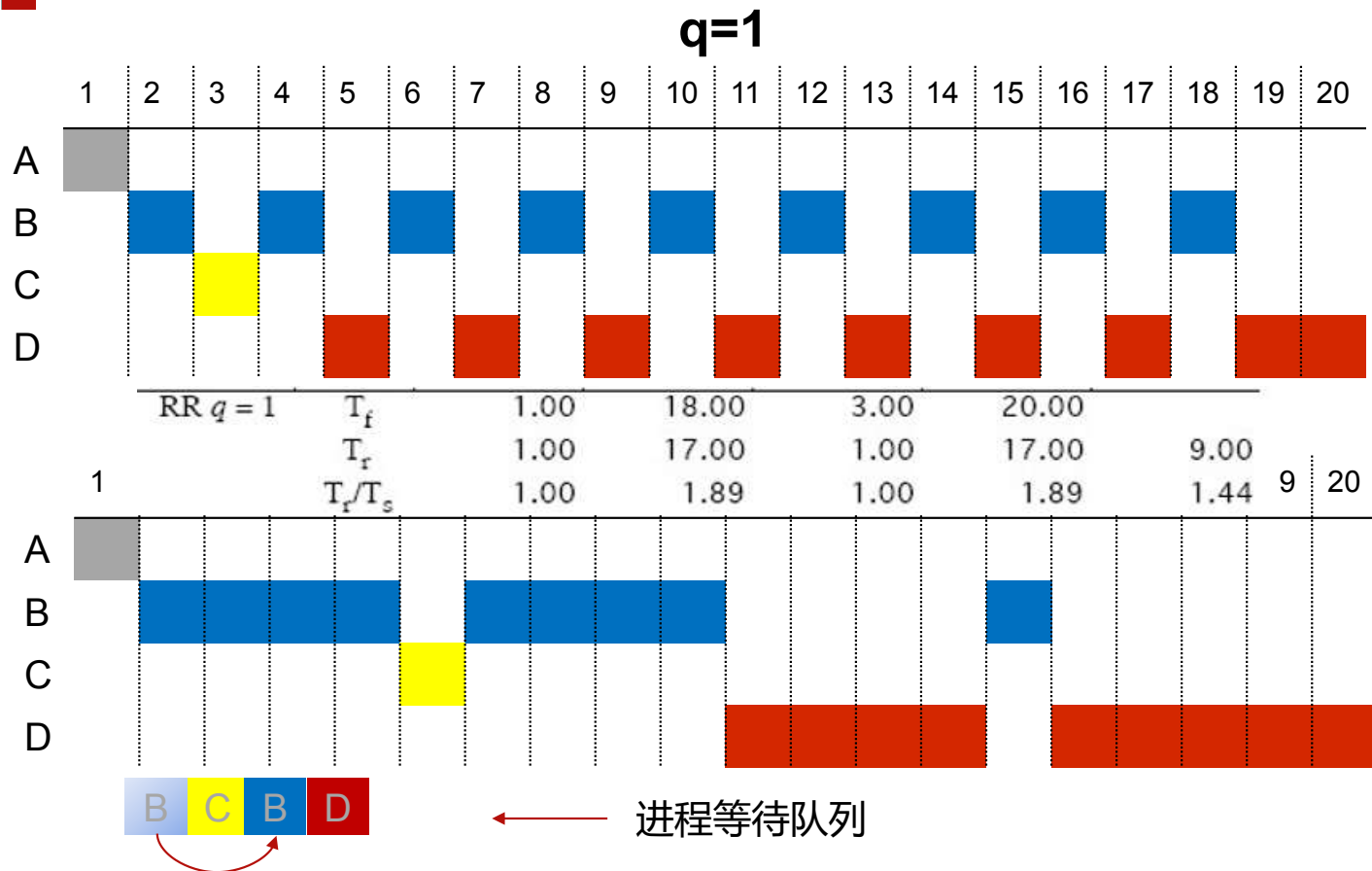
这种算法周期性地产生时钟中断，出现中断时，当前正运行的进程会放置到就绪队列中，然后基于FCFS 策略选择下一个就绪作业运行。

轮转法最主要的设计问题是所用的时间段(片) 长度。

进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9



时间片轮转RR

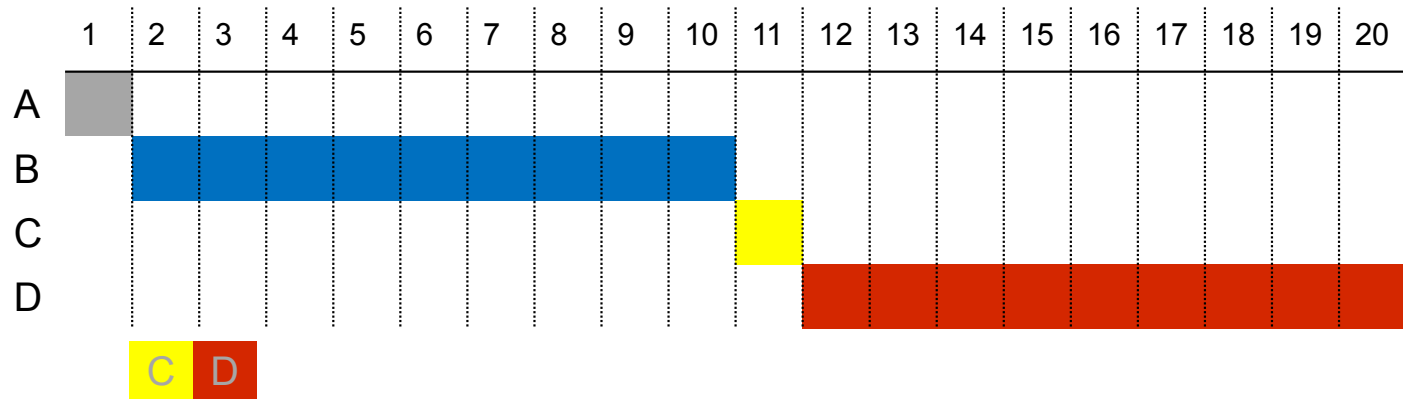


进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9

SPN



最短进程优先，下一次选择所需处理时间最短的进程，非抢占。



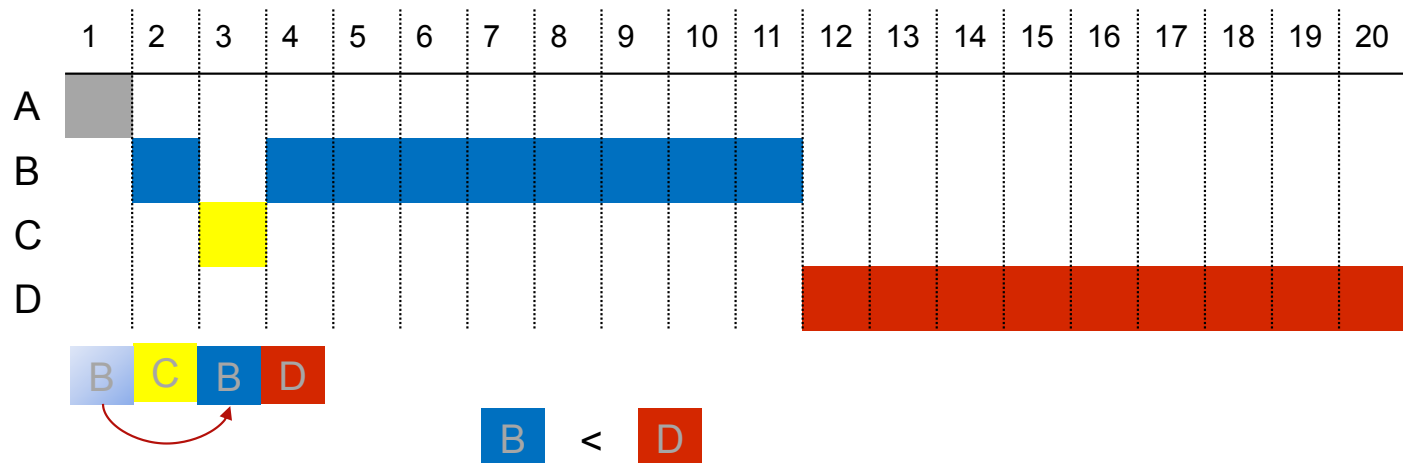
进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9

SPN	T_f	1.00	10.00	11.00	20.00	
	T_r	1.00	9.00	9.00	17.00	9.00
	T_r/T_s	1.00	1.00	9.00	1.89	3.22

SRT



最短剩余时间优先，总是选择预期剩余时间最短的进程，相较于SPN增加了**抢占机制**的策略。



进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9

SRT	T_f	1.00	11.00	3.00	20.00	
	T_r	1.00	10.00	1.00	17.00	7.25
	T_r/T_s	1.00	1.11	1.00	1.89	1.25

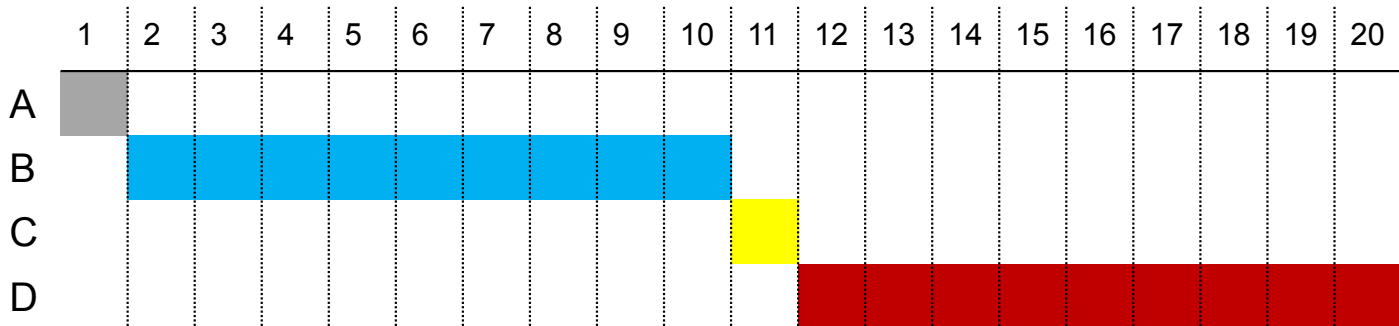
HRRN



最高响应比优先, $R=(w+s)/s$, 其中R表示响应比, w表示已经等待的时间, s表示期待服务的时间

调度规则如下:当前进程完成或被阻塞时, **选择R值最大的就绪进程**。HRRN偏向短作业时(小分母产生大比值), 长进程由于得不到服务, 等待的时间会不断地增加, 因此比值变大, 最终在竞争中赢了

短进程。非抢占式

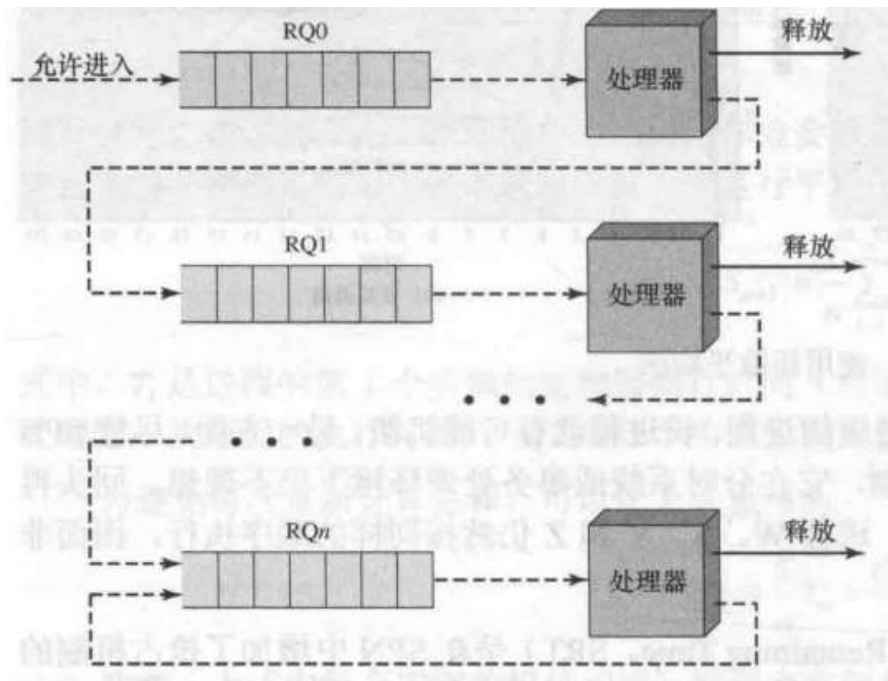


进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9

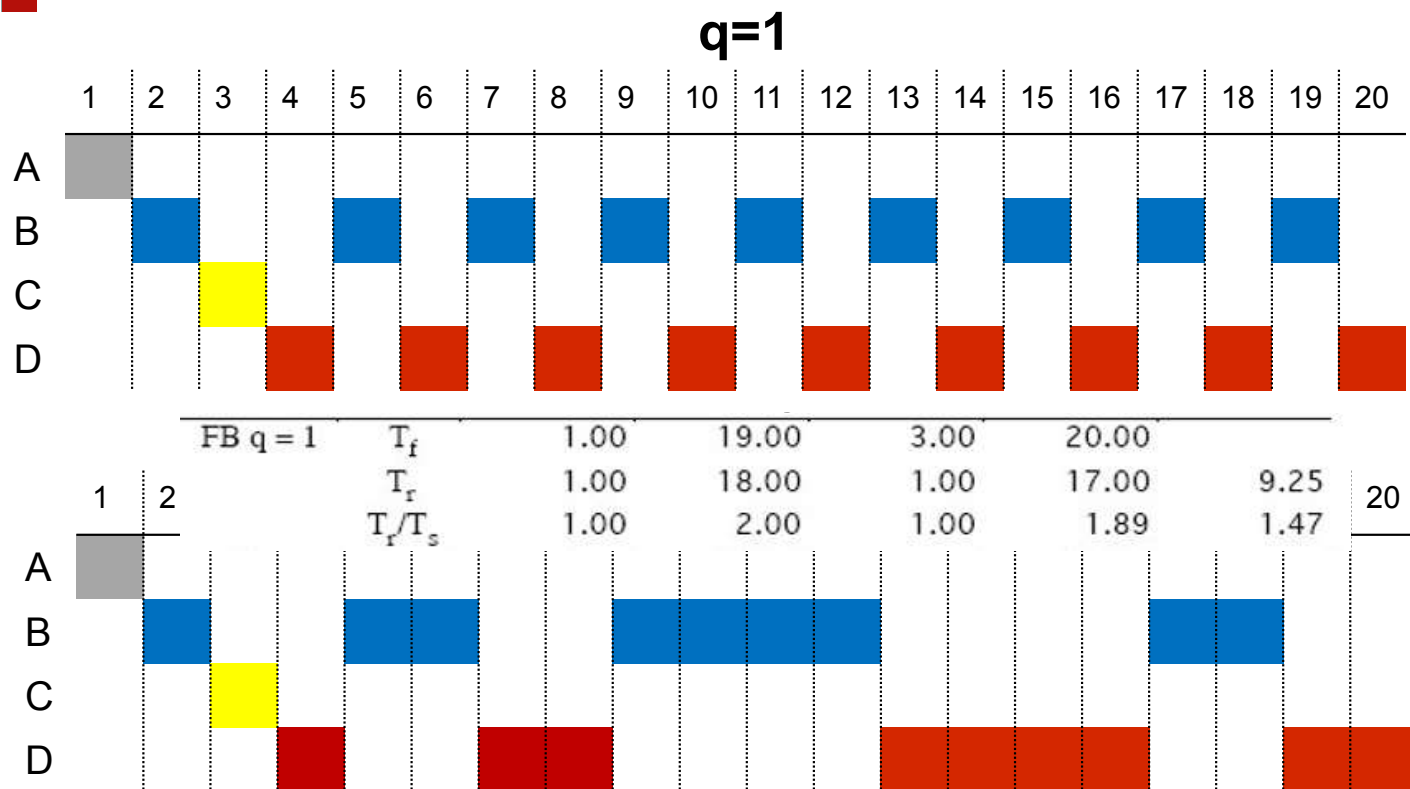


反馈 Feedback

具体方法如下:一个进程首次进入系统中时,会放在RQ0中。当它首次被抢占并返回就绪态时,会放在RQ1中。在随后的时间里, **每当它被抢占时,都降级到下一个低优先级队列中**。短进程很快就会执行完毕,不会出现在就绪队列中多次降级的现象,长进程则会多次降级。因此, **新到的进程和短进程会优先于老进程和长进程**。在每个队列中,除优先级最低的队列外,都使用简单的FCFS机制。进程处于优先级最低的队列中后,就不会再降低,但会重复返回该队列,直到运行结束。



反馈 Feedback



进程	A	B	C	D
到达	0	1	2	3
处理	1	9	1	9

进程调度算法



FCFS

RR, $q = 1$

RR, $q = 4$

SPN

SRT

HRRN

Feedback, $q = 1$

Feedback, $q = 2^i$

A	B	B	B	B	B	B	B	B	B	B	C	D	D	D	D	D	D	D	D
A	B	C	B	D	B	D	B	D	B	D	B	D	B	D	B	D	B	D	D
A	B	B	B	B	C	B	B	B	B	B	C	D	D	D	D	B	D	D	D
A	B	B	B	B	B	B	B	B	B	B	C	D	D	D	D	D	D	D	D
A	B	C	B	B	B	B	B	B	B	B	B	D	D	D	D	D	D	D	D
A	B	B	B	B	B	B	B	B	B	B	C	D	D	D	D	D	D	D	D
A	B	C	D	B	D	B	D	B	D	B	D	B	D	B	D	B	D	B	D
A	B	C	D	B	B	D	D	B	B	B	B	D	D	D	D	B	B	D	D

		A	B	C	D	
	T_a	0	1	2	3	
	T_s	1	9	1	9	
FCFS	T_f	1.00	10.00	11.00	20.00	
	T_r	1.00	9.00	9.00	17.00	9.00
	T_r/T_s	1.00	1.00	9.00	1.89	3.22
RR $q = 1$	T_f	1.00	18.00	3.00	20.00	
	T_r	1.00	17.00	1.00	17.00	9.00
	T_r/T_s	1.00	1.89	1.00	1.89	1.44
RR $q = 4$	T_f	1.00	15.00	6.00	20.00	
	T_r	1.00	14.00	4.00	17.00	9.00
	T_r/T_s	1.00	1.56	4.00	1.89	2.11
SPN	T_f	1.00	10.00	11.00	20.00	
	T_r	1.00	9.00	9.00	17.00	9.00
	T_r/T_s	1.00	1.00	9.00	1.89	3.22
SRT	T_f	1.00	11.00	3.00	20.00	
	T_r	1.00	10.00	1.00	17.00	7.25
	T_r/T_s	1.00	1.11	1.00	1.89	1.25
HRRN	T_f	1.00	10.00	11.00	20.00	
	T_r	1.00	9.00	9.00	17.00	9.00
	T_r/T_s	1.00	1.00	9.00	1.89	3.22
FB $q = 1$	T_f	1.00	19.00	3.00	20.00	
	T_r	1.00	18.00	1.00	17.00	9.25
	T_r/T_s	1.00	2.00	1.00	1.89	1.47
FB $q = 2^i$	T_f	1.00	18.00	3.00	20.00	
	T_r	1.00	17.00	1.00	17.00	9.00
	T_r/T_s	1.00	1.89	1.00	1.89	1.44

进程调度算法



表 9.3 各种调度策略的特点

	FCFS	轮转	SPN	SRT	HRRN	反馈
选择函数	$\max[w]$	常数	$\min[s]$	$\min[s - e]$	$\max\left(\frac{w + s}{s}\right)$	(见正文)
决策模式	非抢占	抢占(时间片用完时)	非抢占	抢占(到达时)	非抢占	抢占(时间片用完时)
吞吐量	不强调	时间片小时,吞吐量会很低	高	高	高	不强调
响应时间	可能很高,尤其是在进程的执行时间差别很大时	为短进程提供较好的响应时间	为短进程提供较好的响应时间	提供较好的响应时间	提供较好的响应时间	不强调
开销	最小	最小	可能较大	可能较大	可能较大	可能较大
对进程的影响	对运行时间短的进程(简称短进程)不利;对 I/O 密集型进程不利	公平对待	对运行时间长的进程(简称长进程)不利	对长进程不利	平衡性好	对 I/O 密集型的进程可能有利
饥饿	无	无	可能	可能	无	可能



四川大學
SICHUAN UNIVERSITY

05

磁盘调度算法

表 11.2 磁盘调度算法比较

(a) FIFO (从磁道 100 处开始)		(b) SSTF (从磁道 100 处开始)		(c) SCAN (从磁道 100 处开始, 以磁道号增大的顺序)		(d) C-SCAN (从磁道 100 处开始, 以磁道号增大的顺序)	
下一个被访问的磁道	横跨的磁道数	下一个被访问的磁道	横跨的磁道数	下一个被访问的磁道	横跨的磁道数	下一个被访问的磁道	横跨的磁道数
55	45	90	10	150	50	150	50
58	3	58	32	160	10	160	10
39	19	55	3	184	24	184	24
18	21	39	16	90	94	18	166
90	72	38	1	58	32	38	20
160	70	18	20	55	3	39	1
150	10	150	132	39	16	55	16
38	112	160	10	38	1	58	3
184	146	184	24	18	20	90	32
	—		—		—		—
平均寻道长度	55.3	平均寻道长度	27.5	平均寻道长度	27.8	平均寻道长度	35.8

例题

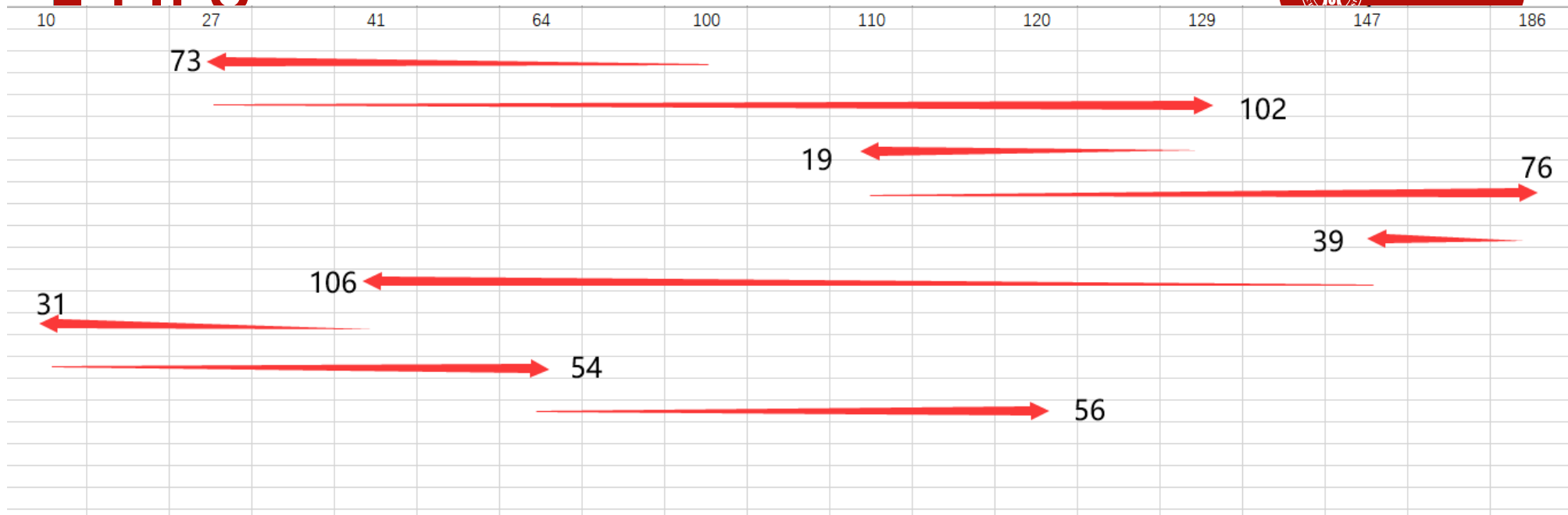


题目：

使用与表11.2类似的方式，分析下列磁道请求：27，129，110，186，147，41，10，64，120。初始磁道为100

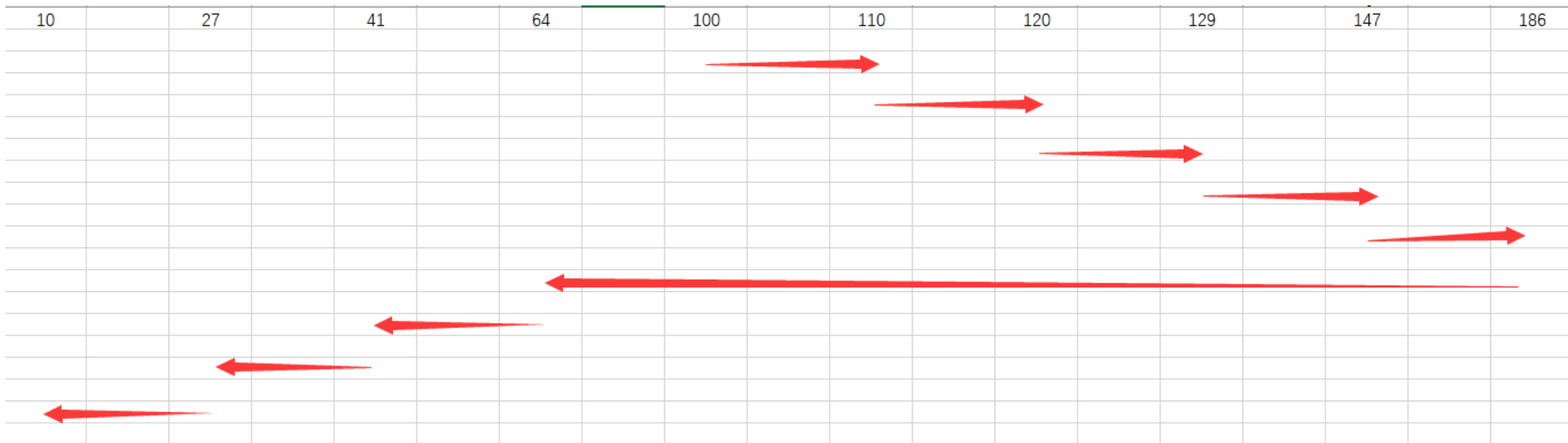
- (1) 磁头沿磁道号减小的方向运行
- (2) 磁头沿磁道号增大的方向运行

FIFO



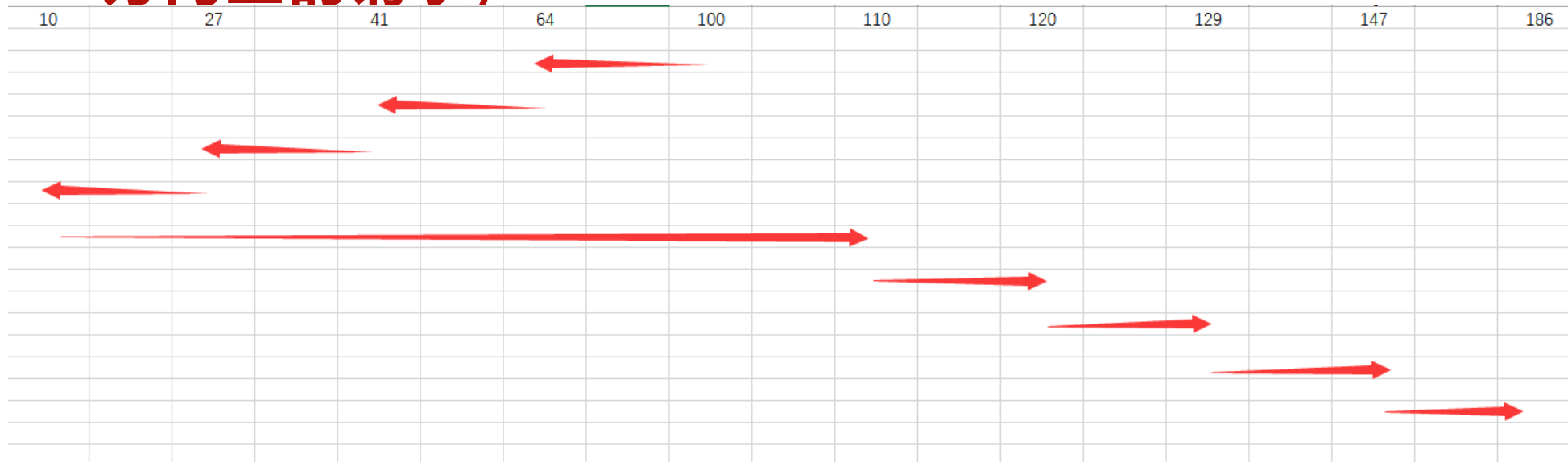
Next	27	129	110	186	147	41	10	64	120	AVG
越过	73	102	19	76	39	106	31	54	56	61.8

SSTF(最短服务时间)



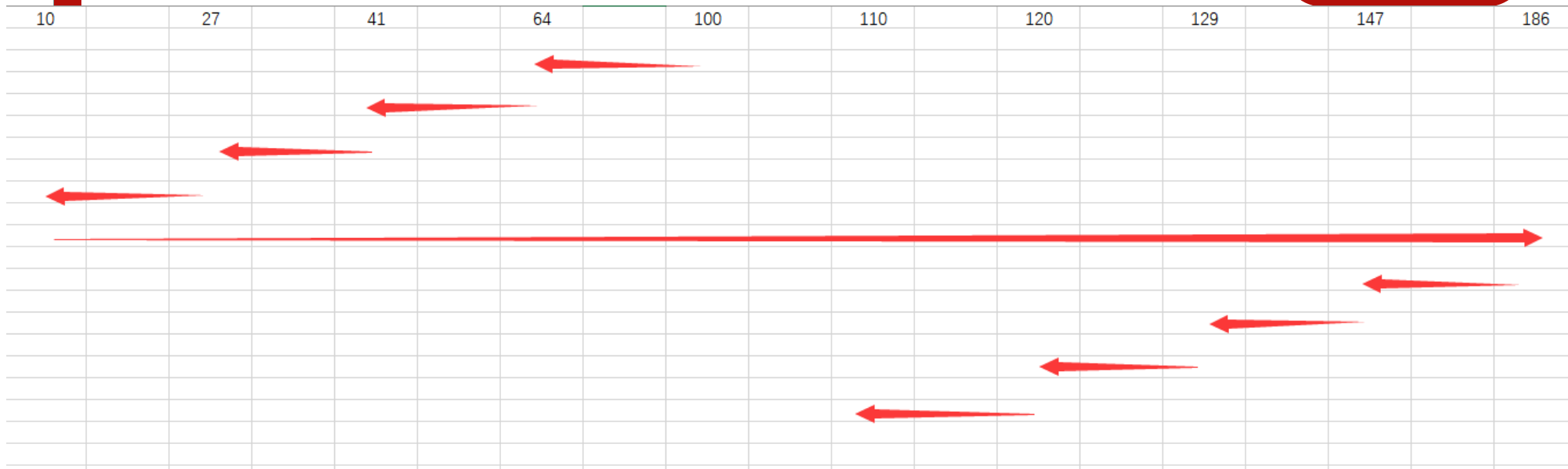
Next	110	120	129	147	186	64	41	27	10	AVG
越过	10	10	9	18	39	122	23	14	17	29.1

SCAN(磁头沿一个方向往返，满足运动方向上的请求)



Next	64	41	27	10	110	120	129	147	186	AVG
越过	36	23	14	17	100	10	9	18	39	29.6

C-SCAN (磁头向一个方向单向运动)



Next	64	41	27	10	186	147	129	120	110	AVG
越过	36	23	14	17	176	39	18	9	10	38

磁盘调度算法



(2) 磁头沿磁道号增大的方向运动
FIFO, SSTF与磁头运动方向无关, 结果与上题一致
SCAN

初始磁道为100

27, 129, 110, 186, 147, 41, 10, 64, 120

Next	110	120	129	147	186	64	41	27	10	AVG
越过	10	10	9	18	39	122	23	14	17	29.6

磁盘调度算法



C-SCAN

初始磁道为100

27, 129, 110, 186, 147, 41, 10, 64, 120

Next	110	120	129	147	186	10	27	41	64	AVG
越过	10	10	9	18	39	176	17	14	23	38